Christie’s East Hampton Platform — Infrastructure Audit

April 6, 2026 · Prepared by Platform Intelligence

Executive Summary

This document is a complete, fresh-eyes audit of every data connection, spreadsheet integration, live feed, caching layer, and evolution mechanism powering the Christie’s East Hampton intelligence platform. It covers the full stack: nine Google Sheets, four live API integrations, six PDF export pathways, the five-layer INTEL architecture, the 35-node Institutional Mind Map, the WhatsApp command system, and all static master data. Where gaps exist — hardcoded values that should be dynamic, missing database tables, or data that has not yet been connected — they are called out explicitly.

Part I — Google Sheets Infrastructure

Authentication Model

All Google Sheets access runs through a single service account: **christies-eh-sheets@christies-hamptons.iam.gserviceaccount.com**. The JSON credentials are stored in the `GOOGLE_SERVICE_ACCOUNT_JSON` environment variable and loaded at runtime in `server/sheets-helper.ts`. Every sheet read and write uses the `googleapis` Node.js library with the `https://www.googleapis.com/auth/spreadsheets` scope. There is no OAuth flow, no user token, and no per-request authentication — the service account acts as a permanent, credential-bearing reader/writer on behalf of the platform.

Critical dependency: If `GOOGLE_SERVICE_ACCOUNT_JSON` is missing or malformed, all four live sheet integrations fail silently and the platform falls back to static data. The error surfaces in the UI as a “sheet not yet shared” message rather than a hard crash.

The Nine Canonical Sheets

All nine sheet IDs are locked in `client/src/pages/tabs/IntelTab.tsx` under the `SHEET_IDS` constant. All nine returned HTTP 200 during this audit (verified April 6, 2026).

#	Name	Sheet ID	Platform Consumer	Access Mode
1	Growth Model v2	`1jR_sO3t7YoKjUlDQpSvZ7hbFNQVg2BD6J4Sqd14z0Ag`	FUTURE tab, INTEL Layer 3	Read via service account
2	Office Pipeline	`1VPjIYPaHXoXQ3rvCn_Wx3nVAUWzM0hBuHhZV92mFz7M`	PIPE tab, INTEL Layer 3	Read + Write via service account
3	Market Matrix	`176OVbAi6PrIVlglnvIdpENWBJWYSp4OtxJ-Ad9-sN4g`	MARKET tab, INTEL Layer 3	Read via service account
4	Future Agents Recruiting	`1a7arxf3_eTAnF7QlD3M-Fwnt7RhOaMWfLlTbA9MJ7mA`	INTEL Layer 3 link only	Link only (no live read)
5	Intelligence Web Master	`1eELH_ZVBMB2wBa9sqQM0Bfxtzu80Am0d21UiIXJpAO0`	INTEL Layer 5, WhatsApp INTEL	Read via service account
6	Social Pipeline	`1q92gJTv1RGX_JGka0KhVv9obePvCOaVdmYjEPuhjc5I`	INTEL Layer 3 link only	Link only (no live read)
7	Event Calendar	`1cBDdmA63ZStEQZLt74WtKU3ewmVaXHWfOgVQhPbOg2s`	INTEL Layer 2 (iframe embed)	Iframe embed only
8	Podcast Calendar	`1mYrrOOcJuKYEdWsDQpY4NNF4I3vO5QW6DhaRXBaRBL8`	INTEL Layer 2 (iframe embed)	Iframe embed only

#	Name	Sheet ID	Platform Consumer	Access Mode
9	Hamptons Outreach Intelligence	`1mEu4wYyWOXit_AIXhOZi9xFQ3y_OklX-fCDMq_i-MlI`	INTEL Layer 3 link only	Link only (no live read)

Three access modes are in use:

1. **Service account read/write** — full API integration, data parsed into typed TypeScript objects, consumed by tRPC procedures, rendered in the platform UI. Sheets 1, 2, 3, and 5 use this mode.
2. **Iframe embed** — the sheet is embedded directly in the browser using Google's `?widget=true&rm=minimal` URL parameter. No server-side parsing. Sheets 7 and 8 use this mode for the Master Calendar panel.
3. **Link only** — the sheet ID is stored in the `SHEET_IDS` constant and the Nine-Sheet Matrix displays a button that opens the sheet in a new tab. No data is pulled into the platform. Sheets 4, 6, and 9 use this mode.

Part II — Live Sheet Data Pipelines

Pipeline A — Office Pipeline → PIPE Tab

Sheet: Office Pipeline (`1VPjIYPaHXoXQ3rvCn_Wx3nVAUWzM0hBuHhZV92mFz7M`) **Tab:** `Sheet1` **Range read:** `Sheet1!A:U` (21 columns) **tRPC procedure:** `trpc.pipe.sheetDeals.useQuery()` **Client refresh:** every 60 seconds (`refetchInterval: 60_000`), stale after 30 seconds (`staleTime: 30_000`)

Column map (A–U):

Column	Field	Notes
A	ADDRESS	Primary key for row lookup
B	TOWN	Hamlet name
C	TYPE	Deal, Listing, Buyer, Seller, etc.
D	PRICE	Ask price
E	STATUS	Written back by platform on status change
F	AGENT	Assigned agent
G	SIDE	Buy-side or sell-side
H	ERS / EBB SIGNED	Engagement agreement status
I	EELI LINK	Christie's listing URL
J	SIGNS	Sign installation status
K	PHOTOS	Photography status
L	ZILLOW SHOWCASE	Zillow showcase status
M–T	Media/marketing fields	Not surfaced in UI
U	DATE CLOSED	Written back by platform on close

Write path: When a user changes a deal status in the PIPE tab, `trpc.pipe.updateSheetStatus.useMutation()` fires, which calls `updatePipelineStatus()` in `sheets-helper.ts`. This writes column E (STATUS) and optionally column U (DATE CLOSED) directly to the Google Sheet. The sheet is the single source of truth — there is no local database mirror of pipeline deals.

Row structure: Row 1 is blank. Row 2 is the column header row. Row 3 onward contains data, with section header rows interspersed (e.g., "BUY-SIDE DEALS" — detected by the parser as all-uppercase with no price value).

Gap: The `isSectionHeader` detection logic is heuristic (all-caps + no price). If a deal address is accidentally entered in all caps, it will be misclassified as a section header and hidden from the UI.

Pipeline B — Market Matrix → MARKET Tab

Sheet: Market Matrix (`176OVbAi6PrIVlglnvIdpENWBJWYSp4OtxJ-Ad9-sN4g`) **Tab:** `Market Matrix` **Range read:** `Market Matrix!A7:K18` (row 7 = headers, rows 8–18 = 11 hamlet data rows) **tRPC procedure:** `trpc.market.hamletMatrix.useQuery()` **Client refresh:** No automatic interval. Data is fetched once on page load and cached by React Query's default stale time.

Column map (A–K):

Column	Field	TypeScript Property
A	Hamlet name	`hamlet`
B	CIS Score	`cisScore` (parsed as float)
C	2025 Median	`median2025`
D	Dollar Volume Share	`dollarVolumeShare`
E	Dollar Volume 2025	`dollarVolume2025`
F	Sales Count 2025	`sales2025` (parsed as int)
G	4-Year Direction	`direction4Year`
H	School District	`schoolDistrict`
I	2022 Median	`median2022`
J	2023 Median	`median2023`
K	2024 Median	`median2024`

Merge logic in MarketTab.tsx: The MARKET tab calls both `trpc.market.hamletMatrix.useQuery()` (live sheet) and reads static data from `hamlet-master.ts`. The `mergeHamletData()` function normalizes hamlet names (e.g., "East Hampton Village" → "EH Village") and merges live values over static fallbacks:

- `liveMedian` = sheet `median2025` if present, else static `medianPriceDisplay`
- `liveCis` = sheet `cisScore` if present (field name `cisScore`), else static `cis` or `anewScore`

- `liveVolumeShare` = sheet `dollarVolumeShare` or `volumeShare` if present, else static value
- `liveDollarVolume`, `liveSales`, `liveDirection` = sheet values if present, else static

Important: The CIS field mapping bug (EH Village showing 0.2 instead of 9.2) was fixed in Sprint 15. The sheet column B is now correctly mapped to `cisScore` and the merge logic reads `cisScore` first. The static fallback field is `cis` (not `anewScore`), which is the correct property name in `hamlet-master.ts`.

Gap: The MARKET tab has no automatic refresh interval. If the Market Matrix sheet is updated during a user's session, they will not see the new data until they reload the page.

Pipeline C — Intelligence Web → INTEL Layer 5

Sheet: Intelligence Web Master (`1eELH_ZVBMB2wBa9sqQM0Bfxtzu80Am0d21UiIXJpAO0`) **Tab:** `Intelligence Web` **Range read:** `Intelligence Web!A:Q` (17 columns, all rows) **tRPC procedure:** `trpc.intel.webEntities.useQuery()` **Client refresh:** No automatic interval. Fetched once on INTEL tab load.

Column map (A–Q):

Column	Field	TypeScript Property
A	Entity Name	`entityName`
B	Entity Type	`entityType` (WHALE, RECRUIT, COMPETITOR, PARTNER, INSTITUTION, MEDIA, ATTORNEY, ADVISOR, COUNCIL)
C	Tier	`tier` (TIER 1, TIER 2, ARCHETYPE, WATCH, ACTIVE, ATTORNEY)
D	Current Firm	`currentFirm`
E	Territory	`territory`
F	Connection 1	`connection1`
G	Connection 2	`connection2`
H	Connection 3	`connection3`
I	Connection Type	`connectionType`
J	Status	`status` (ACTIVE, WARM, COLD, TRACKING)
K	Last Intel Date	`lastIntelDate`
L	Notes	`notes`
M	Owner	`owner`
N	Archetype Match	`archetypeMatch`
O	Audience	`audience` (multi-value: Jarvis_Top_Agents, Whale_Intelligence, Auction_Referrals)
P	Last Touch	`lastTouch`
Q	Cadence	`cadence`

Filter logic: The four tabs in `IntelligenceWebTabs.tsx` filter the same entity array client-side:

- **All Entities** — full table with four filter controls (Type, Tier, Audience, Last Touch text search)
- **Jarvis Top Agents** — `entityType === 'RECRUIT'` AND `tier === 'TIER 1'`
- **Whale Intelligence** — `entityType === 'WHALE'`
- **Auction Referrals** — `entityType === 'PARTNER'` OR `entityType === 'INSTITUTION'`

Gap: The Intelligence Web sheet must be manually shared with the service account (Viewer access) for live data to load. If not shared, the UI displays an instructional error message with the service account email. There is no automated sharing mechanism.

Pipeline D — Growth Model v2 → FUTURE Tab

Sheet: Growth Model v2 (`1jR_sO3t7YoKjUlDQpSvZ7hbFNQVg2BD6J4Sqd14z0Ag`) **Tabs read:** `OUTPUTS!A1:H10`, `ROSTER!A1:Q45`, `ASSUMPTIONS!A1:C10`, `VOLUME!A1:J12` **tRPC procedures:** `trpc.future.volumeData.useQuery()` (5-minute stale time)

The Growth Model is the most complex sheet integration. Four separate tab reads are performed:

- **OUTPUTS tab** — year-by-year GCI projections (2025–2031), agent counts, house take. Row 0 = title, Row 1 = headers, Rows 2–7 = data years.
- **ROSTER tab** — individual agent records with GCI projections per year (2026–2031). Row 0 = title, Row 1 = headers, Row 2+ = agents. Section rows (SUBTOTAL, ENGINE, TOTAL, HOUSE, AGENT COUNT, EXISTING AGENTS) are filtered out.
- **ASSUMPTIONS tab** — model parameters (split percentages, growth rates). Row 0 = title, Row 1 = headers, Row 2+ = data.
- **VOLUME tab** — sales volume projections and actuals per agent per year (2026–2028). This is the governing source for the FUTURE tab agent table and Ascension Arc chart. The `parseDollar()` function strips `$`, `,`, `TBD`, and whitespace before parsing.

Gap: The FUTURE tab uses `staleTime: 5 * 60 * 1000` (5 minutes), meaning the arc chart and agent table will show data up to 5 minutes stale. There is no manual refresh button on the FUTURE tab.

Part III — External API Integrations

Perplexity API — Mind Map Hover News

Endpoint: `https://api.perplexity.ai/chat/completions` **Model:** `sonar` **tRPC procedure:** `trpc.intel.entityNews.useQuery({ entityName })` **Trigger:** Fires when a user hovers over a Mind Map node (`hoveredNodeName` becomes non-null) **Client-side cache:** `staleTime: 5 * 60 * 1000` (5 minutes via React Query) **Server-side cache:** None. Each request goes directly to Perplexity with a 15-second abort timeout. **Prompt:** "What is the most significant news about [entityName] in the last 30 days? Real estate, business, or professional news only. Named sources only." **Response handling:** If Perplexity returns "No recent news." the panel shows nothing. If the API is unavailable, the panel shows a silent error state.

Important distinction: The 5-minute cache is client-side only (React Query's in-memory cache). If the user refreshes the page or opens the platform in a new tab, the cache is empty and a fresh Perplexity call fires on the next hover. There is no persistent server-side cache for news responses.

Yahoo Finance + FRED + CoinGecko — Market Data Strip

Route: `GET /api/market-data` (Express route in `server/market-route.ts`) **Data points fetched:**

- S&P 500 — Yahoo Finance chart API
- Gold spot price — Yahoo Finance
- Silver spot price — Yahoo Finance
- VIX — Yahoo Finance
- 30-Year Treasury yield — Yahoo Finance
- Bitcoin — CoinGecko API
- 30-Year mortgage rate — FRED CSV series `MORTGAGE30US`

Caching: The mortgage rate has a 24-hour in-memory cache (`mortgageCache`). All other values are fetched fresh on each request. **Client refresh:** `DashboardLayout.tsx` calls this endpoint every 5 minutes. **Display:** The five-layer header strip shows S&P,

Bitcoin, mortgage rate, gold, silver, VIX, and Treasury. The Hamptons Median displayed in the header (`$2.34M`) is **static** — manually updated per market report cycle, not pulled from any API.

ElevenLabs — WhatsApp Voice Synthesis

Endpoint: ElevenLabs TTS API **Voice ID:** `fjnwTZkKtQOJaYzGLa6n` (William voice — locked April 6, 2026 morning audit) **Trigger:** WhatsApp NEWS command **Cache:** 6-hour in-memory audio URL cache. If the same NEWS request arrives within 6 hours, the cached S3 audio URL is returned without re-generating. **Storage:** Generated audio is uploaded to S3 via `storagePut()` and the URL is sent back to the WhatsApp user.

Twilio — WhatsApp Inbound Webhook

Webhook: `POST /api/whatsapp/inbound` **Commands handled:** NEWS, PIPE, STATUS, BRIEF, INTEL, HELP, and address-specific BRIEF (e.g., "26 Park Place") **Data consumed:**

- NEWS → Perplexity 14-category brief + ElevenLabs voice synthesis
- PIPE → `readPipelineDeals()` from Office Pipeline sheet (top 5 active deals)
- INTEL → `readIntelWebRows()` from Intelligence Web sheet (top 5 Jarvis recruits + top 3 whales)
- STATUS → Hardcoded "47 entities tracked" (not live from sheet)
- BRIEF [address] → Static hamlet lookup table with hardcoded medians and CIS values + LLM positioning paragraph

Gap: The STATUS command reports "47 entities tracked" as a hardcoded string. The actual entity count in the Intelligence Web sheet may differ. This should be replaced with a live `readIntelWebRows().length` call.

Gap: The BRIEF [address] command uses a hardcoded hamlet lookup table inside `whatsapp-inbound.ts` with static medians and CIS scores. These values are not connected to the Market Matrix sheet. If the Market Matrix is updated, the WhatsApp BRIEF responses will not reflect the change.

Part IV — Static Master Data

hamlet-master.ts

Location: `shared/hamlet-master.ts` **Purpose:** Single source of truth for all 11 hamlet static properties **Fields per hamlet:**

Field	Description	Live or Static
`id`	Hamlet slug	Static
`name`	Display name	Static
`imageUrl`	CDN photo URL	Static (CDN)
`photo`	CDN photo URL (alias)	Static (CDN)
`vibeText`	Council-approved description	Static
`cis`	Christie's Intelligence Score (fallback)	Static
`anewScore`	Legacy CIS alias	Static
`medianPriceDisplay`	Formatted median price (fallback)	Static
`lastSale`	Most recent notable sale	Static
`lastSaleHidden`	Hide last sale if data unavailable	Static flag
`restaurants`	Three-tier restaurant list	Static
`eeleListings`	EELE listing placeholders	Static

All 11 hamlet photos are hosted on the Manus CDN (`files.manuscdn.com`) and were uploaded from Ed Bruehl's original photography in Sprint 15. The `imageUrl` and `photo` fields point to the same CDN URL for each hamlet.

Fallback role: When the Market Matrix sheet is unavailable or a hamlet name cannot be matched, the MARKET and MAPS tabs fall back entirely to `hamlet-master.ts` values. The merge logic in `MarketTab.tsx` uses `mergeHamletData()` which normalizes names and overlays live values on top of static ones.

Part V — INTEL Tab Architecture (Five Layers)

Layer 1 — Institutional Mind Map

Component: `InstitutionalMindMap.tsx` **Data source:** Entirely static — the 35-node `NODES` array and `CONNECTIONS` array are hardcoded in the component file. **Live element:** Hover news panels via `trpc.intel.entityNews.useQuery({ entityName })` (Perplexity, 5-minute client cache) **Visualization:** SVG canvas, 1280×1050px, two parallel tracks (Auction House/Family left, Real Estate Operating right), Ed Bruehl as central bridge node at (640, 730)

Node inventory (35 nodes):

Track	Nodes
Crown	Artémis S.A.
Left — Auction House / Family	François-Henri Pinault, Bonnie Brennan, Tash Perrin, Stephen Lash, Julien Pradels, Rahul Kadakia, David Gooding
Right — Real Estate Operating	CIH/Robert Reffkin, Thad Wong/Mike Golden, Gavin Swartzman, Ilija Pavlovic, Melissa True, Sherri Balassone
Center Bridge	Ed Bruehl
Christie's EH Team	Jarvis Slade Jr., Zoila Ortega Astor, Angel Theodore, Sebastian Mobo
Whales	Lily Fan, Rick Moeser, Tony Ingrao, Josh Schnepps, Heath Freeman, Art Murray
Recruits	Marilyn Clark, Debbie Brenneman, Charlie Esposito, Michael Esposito, Nola Baris
Relationship Intelligence	Frank Newbold
Attorneys	Pierre Debbas, Brian Lester, Jonathan Tarbet, Seamus McGrath

Evolution mechanism: To add a new node, a developer edits the `NODES` array in `InstitutionalMindMap.tsx` and adds the corresponding entry to `CONNECTIONS`. There is no admin UI or sheet-driven node management. This is intentional for institutional stability — the mind map reflects curated relationships, not a live feed.

Layer 2 — Master Calendar

Component: Inline iframe embeds in `IntelTab.tsx` **Data source:** Two Google Sheets embedded via `?widget=true&rm=minimal&headers=false` URL parameters

- Left panel: Podcast Calendar (`1mYrrOOcJuKYEdWsDQpY4NNF4I3vO5QW6DhaRXBaRBL8`)
- Right panel: Event Calendar (`1cBDdmA63ZStEQZLt74WtKU3ewmVaXHWfOgVQhPbOg2s`)

Refresh: Google's iframe embed refreshes automatically when the sheet is updated. No server-side caching or parsing. The "Open Google Calendar ↗" link points to `https://calendar.google.com/calendar/r` (generic Google Calendar, not a specific calendar).

Gap: The Google Calendar link is generic, not pointing to a specific Christie's East Hampton calendar. If a shared Google Calendar exists, this link should be updated to point to it directly.

Layer 3 — Nine-Sheet Matrix

Component: `NineSheetMatrix()` function in `IntelTab.tsx` **Data source:** Static `NINE_SHEETS` array — nine entries with sheet IDs, names, descriptions, and badge labels **Interaction:** Each entry renders an "Open Sheet ↗" button that opens the sheet in a new tab via `sheetOpenUrl(id)`

All nine sheet IDs confirmed live (HTTP 200) as of April 6, 2026. The matrix is a navigation layer, not a data display layer — it provides institutional awareness of what each sheet contains and which platform surface it drives.

Layer 4 — Document Library

Component: `DocumentLibrary()` function in `IntelTab.tsx` **Data source:** Static `DOCUMENT_LIBRARY` array — six entries with CDN/CloudFront URLs

Document	URL Type	Status
CIREG Ecosystem Org Chart (April 2, 2026)	CloudFront HTML	✓ Live (HTTP 200)
Estate Advisory Card PDF	CloudFront PDF	✓ Live (HTTP 200)
300-Day Ascension Plan	Manus CDN HTML	✓ Live (HTTP 200)
Christie's Hamptons Live Market Report v2	Manus CDN HTML	✓ Live (HTTP 200)
Council Brief March 29, 2026	Manus CDN HTML	✓ Live (HTTP 200)
Modern Day Path · UHNW Backend Strategy	CloudFront PDF	✓ Live (HTTP 200)
Christie's Intelligence Web Canonical Map	CloudFront HTML	✓ Live (HTTP 200)

Note: The Manus CDN URLs (`files.manuscdn.com/user_upload_by_module/session_file/...`) are session-file uploads and may not persist indefinitely. The CloudFront URLs (`d2xsxph8kpxj0f.cloudfront.net`) are webdev-static CDN assets with the same lifecycle as the deployed project and are the more durable format.

Gap: The April 6, 2026 council brief is not yet in the Document Library. The most recent brief shown is March 29, 2026.

Layer 5 — Intelligence Web (Relationship Intelligence)

Component: `IntelligenceWebTabs.tsx` **Data source:** `trpc.intel.webEntities.useQuery()` → `readIntelWebRows()` → Intelligence Web Master sheet **Four tabs:** All Entities, Jarvis Top Agents, Whale Intelligence, Auction Referrals **Filter controls (All Entities tab):** Type dropdown, Tier dropdown, Audience dropdown, Last Touch text search **Entity count:** 47–48 entities (live from sheet; UI displays "48 entities" in the layer header)

The four tabs share a single data fetch. All filtering is client-side. The "Open Sheet ↗" button links directly to the Intelligence Web Master sheet for editing.

Part VI — PDF Export Infrastructure

All six PDF export functions are client-side, using the `jsPDF` library. There is no server-side PDF generation in production (Puppeteer was deprecated in Sprint 15g due to Chromium dependency failures).

Export	Function	Trigger	Status
Market Report	`generateMarketReport()`	REPORT page button	✓ Working (migrated from Puppeteer in Sprint 15g)
Hamlet PDF (11 variants)	`generateHamletReport(hamlet)`	MARKET tab per-hamlet button	✓ Working
Christie's Letter	`generateChristiesLetter()`	HOME tab button	✓ Working
Deal Brief	`generateDealBrief(deal)`	PIPE tab per-deal button	✓ Working
CMA	`generateCMA(data)`	PIPE tab CMA button	✓ Working
Investment Memo	`generateInvestmentMemo(data)`	PIPE tab memo button	✓ Working
Future Report	`generateFutureReportPDF()`	FUTURE tab button	✓ Working

Christie's Letter QR codes: Two QR codes are embedded in the Christie's Letter PDF using the `qrcode` npm package:

- Website QR: `christiesrealestategroupeh.com`
- Ed vCard QR: encoded vCard 3.0 with Ed Bruehl's contact information Both are rendered as 22×22mm base64 PNG images on a navy background with cream-colored modules.

Image loading: All PDF exports that include hamlet photos load images from the Manus CDN via `fetch()` at generation time. If the CDN is unavailable, the image is skipped and the PDF generates without it (no hard failure).

Part VII — Database Schema

The PostgreSQL database has two tables:

Table	Purpose	Used by
`users`	OAuth authentication — openId, name, email, role, timestamps	Auth system
`pipeline`	Local pipeline record (address, hamlet, type, status, askPrice, dom, notes)	Not actively used — Office Pipeline sheet is the source of truth

Critical gap: The `pipeline` database table exists but is not the active data store for deals. The PIPE tab reads and writes exclusively to the Google Sheet. The database table is a legacy scaffold from the initial template and has not been removed. This creates a potential confusion point — any code that writes to the `pipeline` table would not be reflected in the PIPE tab UI.

Missing tables: There are no database tables for:

- Intelligence Web entities (all entity data lives in the Google Sheet)
- WhatsApp conversation history or threading
- Institutional relationship metadata
- Perplexity news cache (all caching is in-memory, lost on server restart)
- Audit logs or change history

Part VIII — Evolution Mechanisms

How Market Data Stays Current

The Market Matrix sheet is the single source of truth for all 11 hamlet metrics. When Ed or the council updates a CIS score, median price, or sales count in the sheet, the MARKET tab will reflect the change on the next page load (no server restart required). The merge logic in `MarketTab.tsx` always prefers live sheet values over static `hamlet-master.ts` fallbacks.

To update market data: Edit the Market Matrix sheet directly. The platform reads it on every page load.

How Intelligence Web Entities Grow

New entities are added by editing the Intelligence Web Master sheet. The platform reads all rows from column A to Q on every INTEL tab load. Adding a row to the sheet automatically adds the entity to all four tabs in Layer 5.

To add a new entity: Add a row to the Intelligence Web sheet with the correct column values. The entity will appear in the All Entities tab immediately and in the filtered tabs if its Type/Tier/Audience values match the filter criteria.

How the Mind Map Grows

The Mind Map does not read from any sheet. New nodes require a code change to `InstitutionalMindMap.tsx`. This is intentional — the mind map represents curated institutional relationships that should be reviewed before being added to the visualization.

To add a new node: Edit the `NODES` array in `InstitutionalMindMap.tsx`, add the `CONNECTIONS` entry, and deploy. The hover news panel will automatically work for the new node (Perplexity queries by `entityName`).

How Pipeline Deals Evolve

The Office Pipeline sheet is the single source of truth. The PIPE tab reads it every 60 seconds. Status changes made in the PIPE tab UI are written back to the sheet immediately. New deals can be added to the sheet directly or via the `appendPipelineRow()` function (currently not exposed in the UI — would require a "New Deal" form).

How the Growth Model Evolves

The Growth Model v2 sheet drives the FUTURE tab. When the ROSTER or VOLUME tab is updated with new agent data or actual sales figures, the FUTURE tab arc chart and agent table will reflect the change within 5 minutes (React Query stale time). The milestone targets (`MILESTONE_TARGETS` in `FutureTab.tsx`) are hardcoded and require a code change to update.

Part IX — Identified Gaps and Recommendations

High Priority

Gap	Location	Impact	Recommendation
WhatsApp STATUS command hardcodes “47 entities”	`whatsapp-inbound.ts`	Stale entity count reported to WhatsApp users	Replace with `readIntelWebRows().length`
WhatsApp BRIEF command uses hardcoded hamlet medians/CIS	`whatsapp-inbound.ts`	BRIEF responses won’t reflect Market Matrix updates	Connect to `readMarketMatrixRows()`
MARKET tab has no auto-refresh	`MarketTab.tsx`	Sheet updates during a session are invisible	Add `refetchInterval: 5 * 60 * 1000`
Perplexity news has no server-side cache	`routers.ts`	Every hover fires a fresh API call; no persistence across restarts	Add in-memory Map cache with 5-minute TTL on the server
`pipeline` database table is unused but present	`drizzle/schema.ts`	Confusion risk — code that writes to DB won’t appear in PIPE tab	Either remove the table or document it clearly as deprecated

Medium Priority

Gap	Location	Impact	Recommendation
Google Calendar link is generic	`IntelTab.tsx`	Users click “Open Google Calendar” and land on their personal calendar	Update to a specific Christie’s EH shared calendar URL
April 6 council brief not in Document Library	`IntelTab.tsx`	Most recent brief is March 29	Add the April 6 brief URL to `DOCUMENT_LIBRARY`
Manus CDN session-file URLs may not persist	`IntelTab.tsx`	Three document links use session-file CDN format	Re-upload to webdev CDN (`manus-upload-file --webdev`) for permanent URLs
EH North last sale hidden	`hamlet-master.ts`	No last sale displayed for EH North	Populate with a real reference sale when available
Restaurant data missing	`hamlet-master.ts`	Restaurant tier fields are empty for most hamlets	Populate from Perplexity research (on hold)

Low Priority / Future Consideration

Gap	Location	Impact	Recommendation
No WhatsApp conversation history in DB	Schema	Cannot review past WhatsApp interactions	Add `whatsapp_messages` table with inbound/outbound, command, response, timestamp
No audit log for sheet writes	`sheets-helper.ts`	No record of who changed what in the pipeline	Add a `pipeline_changes` table logging address, old status, new status, timestamp
Mind Map node management requires code deploy	`InstitutionalMindMap.tsx`	Adding a node requires a developer	Consider a sheet-backed node config for non-developer updates
INTEL Layer 5 has no auto-refresh	`IntelligenceWebTabs.tsx`	Sheet updates during session are invisible	Add `refetchInterval: 5 * 60 * 1000`

Part X — Data Flow Summary

```
Google Sheets (9 sheets)
  ├── Office Pipeline ──────────────────────────────→ PIPE Tab
(read/write, 60s refresh)
  ├── Market Matrix ────────────────────────────────→ MARKET Tab (read,
page-load only)
  ├── Intelligence Web Master ──────────────────────→ INTEL Layer 5
(read, page-load only)
  │                                                 → WhatsApp INTEL
command
  ├── Growth Model v2 ──────────────────────────────→ FUTURE Tab (read,
5-min stale)
  ├── Podcast Calendar ─────────────────────────────→ INTEL Layer 2
(iframe embed)
  ├── Event Calendar ───────────────────────────────→ INTEL Layer 2
(iframe embed)
  └── Future Agents, Social Pipeline, Outreach Intel ──→ INTEL Layer 3 (link
only)

External APIs
  ├── Perplexity (sonar) ───────────────────────────→ Mind Map hover
news (on-demand)
  │                                                 → WhatsApp NEWS
command (daily brief)
  ├── Yahoo Finance + FRED + CoinGecko ─────────────→ Dashboard header
strip (5-min refresh)
  ├── ElevenLabs TTS ───────────────────────────────→ WhatsApp NEWS
audio (6-hr cache)
  └── Open-Meteo ───────────────────────────────────→ Dashboard weather
widget

Static Data
  ├── hamlet-master.ts ─────────────────────────────→ MARKET Tab
(fallback)
  │                                                 → MAPS Tab
  │                                                 → PDF exports
  ├── InstitutionalMindMap.tsx (NODES + CONNECTIONS) ──→ INTEL Layer 1
(static + Perplexity hover)
  └── DOCUMENT_LIBRARY + NINE_SHEETS constants ─────→ INTEL Layers 3 + 4

PDF Exports (client-side jsPDF)
  ├── Market Report ────────────────────────────────→ REPORT page
  ├── Hamlet PDFs (×11) ────────────────────────────→ MARKET tab
  ├── Christie's Letter ────────────────────────────→ HOME tab
```

```
    ├── Deal Brief, CMA, Investment Memo ──────────────────→ PIPE tab
    └── Future Report ──────────────────────────────────────→ FUTURE tab
```

Conclusion

The Christie's East Hampton platform is a well-structured intelligence system with four live Google Sheets integrations, four external API connections, and a comprehensive static data layer that provides reliable fallbacks. The architecture is sound: Google Sheets serve as the operational source of truth for deals, market data, and entity intelligence; the platform reads and writes to them via a service account without requiring user authentication.

The most significant gaps are operational rather than architectural: the WhatsApp BRIEF and STATUS commands use hardcoded values that will drift from the live sheet data over time, the MARKET and INTEL tabs lack auto-refresh intervals, and the Perplexity news cache is ephemeral. None of these gaps affect the core platform functionality or the client-facing experience. They are refinements that would move the system from "reliable" to "fully automated."

The platform is production-ready as of Sprint 15 Final (checkpoint `31f5c370`). All six PDF exports work. All nine sheets are live. All 35 Mind Map nodes are correctly positioned. All 11 hamlet photos are on CDN. The Document Library has six active documents. The Intelligence Web has 47–48 entities accessible through four filtered views.

Audit completed April 6, 2026. All sheet IDs and CDN URLs verified live at time of writing.